

DroidPR

AI Android Pull Request Reviewer

A focused, deployable GenAI engineering project for a final-year CSE portfolio

Product GitHub App + web dashboard for Android engineering teams	Scope Security, accessibility and dependency-risk reviews
Core promise Evidence first. Human approved. Never a generic chatbot.	Target duration 5 focused weeks, around 15-20 hours per week

The goal is not to replace Android engineers. DroidPR turns noisy scanner output into a review that an engineer can trust, understand and act on.

1. The problem

Android pull requests are difficult to review well. A reviewer must reason across Kotlin, Jetpack Compose or XML, AndroidManifest.xml, Gradle dependencies and platform behavior. Security, accessibility and dependency regressions often pass code review because they are easy to miss and expensive to inspect manually.

Existing scanners are useful but fragmented: one flags a pattern, another detects a vulnerable dependency, and another runs accessibility checks. Developers still need to decide whether the issue is real, why it matters, where to fix it, and whether a proposed change is safe.

- **Core pain:** Android teams receive either too little review coverage or too many low-context warnings.
- **Why generic AI review is insufficient:** it can hallucinate issues and lacks Android-specific evidence.
- **Who benefits:** Android developers, student teams, startups and open-source maintainers using GitHub.

2. The solution

DroidPR is a web-based GitHub App that reviews Android pull requests. It combines deterministic scanners with an agentic reasoning layer. The scanners provide evidence; the AI explains, prioritizes and proposes a narrowly scoped repair. A human remains in control of every code change.

- **Evidence-first:** findings must link to a changed file, code location and scanner/tool result.
- **Android-native:** understands Kotlin, Compose, XML, Manifest and Gradle rather than treating the repository as generic text.
- **Safe by design:** no direct merging; generated fixes are drafts and need approval.
- **Measurable:** the dashboard tracks precision, review latency, false positives and patch validation success.

3. Product workflow

1. Install Team installs DroidPR GitHub App and selects repositories.	2. Trigger A pull request is opened or updated.	3. Collect Webhook fetches diff and Android project context.
4. Scan Run Android Lint, Semgrep rules, dependency scan and accessibility checks.	5. Reason Agent groups confirmed signals, retrieves guidance and scores confidence.	6. Review Post a concise GitHub summary and inline comments.
7. Fix preview Developer asks for a patch only for selected findings.	8. Validate Run lint, unit tests or Gradle build in an isolated worker.	9. Approve Human accepts a draft PR; dashboard stores trace and outcome.

Important rule: DroidPR can recommend and prepare a patch, but it never silently pushes or merges code.

4. What DroidPR checks

- **Security:** exported components, insecure WebView configuration, cleartext traffic, exposed secrets and unsafe storage patterns.
- **Accessibility:** missing content descriptions, risky touch-target sizes, contrast and semantics problems, especially in Compose and XML UI changes.
- **Dependencies:** vulnerable or deprecated dependencies, license concerns and upgrade recommendations.
- **Privacy review prompts:** permissions and SDK changes that deserve a human check before release.

5. Agent design

The agent is deliberately bounded. It is not asked to invent vulnerabilities from a whole repository. It receives changed code, structured scanner signals and retrieved official guidance. It returns a strict schema: finding, evidence, severity, confidence, explanation, recommendation and optional patch.

A critic stage rejects claims with no evidence, duplicate findings and patches that fail validation. The product therefore demonstrates useful GenAI engineering: tool calling, workflow orchestration, retrieval, guardrails, observability and evaluation.

6. Website experience

- **Landing page:** clear product story, architecture illustration, public demo repository and security model.
- **Authentication:** GitHub OAuth; users install the GitHub App and choose a repository.
- **Dashboard:** review history, finding categories, severity trends, run status and validated patch outcomes.
- **Pull request view:** one readable summary, inline evidence, confidence labels, source guidance and a “Generate fix preview” action.
- **Evaluation page:** transparent benchmark results and known limitations - excellent interview material.

7. Technical architecture

Frontend Next.js, TypeScript, Tailwind, GitHub OAuth	Backend FastAPI, webhook API, Celery/RQ workers, PostgreSQL
AI workflow LangGraph, structured outputs, retrieval over Android and OWASP guidance, tracing	Tools Android Lint, Semgrep, OSV-Scanner, Gradle test/build in Docker
Deployment Vercel for web; Railway/Render/Fly.io for API and workers; managed Postgres	Safety Ephemeral isolated workspaces, least-privilege GitHub tokens, approval before mutation

8. Five-week delivery plan

Week	Goal	Deliverable
1	Foundation	FastAPI + Postgres, GitHub OAuth/App, webhook receiver, repository and PR data model.
2	Deterministic review engine	Android Lint, Semgrep and OSV scanner adapters; normalized finding schema; basic PR summary.
3	Grounded AI layer	LangGraph workflow, official-guidance retrieval, evidence-first explanations, confidence thresholds and traces.
4	Website and safe fix flow	Next.js dashboard, scan history, inline review UX, draft patch preview and approval model.
5	Validation and launch	Docker validation worker, benchmark suite, metrics page, domain deployment, demo video and README.

Realistic commitment: 75-100 focused hours. At 15-20 hours a week, this is about five weeks. If you work only 8-10 hours a week alongside DSA and system design, plan six to eight weeks.

9. Definition of done

- A public website on your domain, with a polished landing page and authenticated dashboard.
- A working GitHub App that reviews a public Android demo repository.
- At least 15 deliberately seeded benchmark cases with reported results.
- A safe, evidence-backed fix preview for a small number of deterministic finding types.
- A 2-3 minute demo video and an honest README that credits all upstream open-source tools.

10. Interview narrative

“I noticed that Android PR reviews require context across platform configuration, UI accessibility and dependencies. I built DroidPR to combine deterministic analysis with LLM reasoning, but deliberately restricted the model to evidence-backed findings and human-approved changes. I measured its behavior on a benchmark suite rather than showing only a happy-path demo.”